

第29回 実践：Web ゲームプログラミング 8（予備・調整）

実習時のフォルダ構成

Web_名前 の中に 29 フォルダを作る。第 28 回の `sample` フォルダと `exercise` フォルダを、29 の中にコピーして、続きから作る。

```
Web_名前
└── 29
    ├── sample          ← 第 28 回の sample をコピーする（今回は書き足さない）
    │   ├── index.html
    │   └── main.js
    └── exercise        ← 第 28 回の exercise をコピーする
        ├── index.html
        └── main.js
```

- `sample`：講義で使うサンプル。第 28 回で完成しているので、今回は `sample/main.js` にコードを書き足さない。
- `exercise`：自分のシューティングゲーム（演習 1～3、発展）

今回は、次の 2 つを目的にする。

- これまでの回で終わっていない部分を仕上げて、自分のゲームを第 28 回の状態まで追いつかせる。
- うまく動かないときの調べ方と、見た目の調整を学ぶ。

講義内容

1. 最終必須仕様と、学んだ回

次の 9 項目が、最終回（第 30 回）までに自分のゲームで満たす「最終必須仕様」。どれも第 28 回までに学んでいる。

No.	仕様	学んだ回	sample/main.js で担当している部分
1	Canvas に自機を表示	第 21 回・第 22 回	drawPlayer
2	キーで左右移動	第 23 回	キー入力、updatePlayer
3	キーで弾を発射	第 24 回	キー入力、updateBullets
4	敵が出現して移動	第 25 回	updateEnemies
5	弾が敵に当たる	第 26 回	isHit、checkBulletHits
6	当たると敵が消えて得点が増える	第 26 回・第 27 回	checkBulletHits、drawUI
7	敵などとの接触で残機が減る	第 27 回	checkPlayerHit、drawUI
8	残機 0 でゲームオーバー	第 28 回	checkPlayerHit、drawGameOver
9	リスタートできる	第 28 回	キー入力 (Enter)、resetGame

- 自分のゲームで、できていない項目があれば、その回のレジメと sample/main.js を見て追加する (演習 1)。

2. うまく動かないときの調べ方

(1) コンソールのエラーを読む

F12 の開発者ツールのコンソールに、赤字でエラーが表示される。エラーの右側には、エラーが起きたファイル名と行番号 (例: main.js:85) が表示される。まず、その行を見る。

よく出るエラーと、主な原因:

エラーの表示 (一部)	主な原因
ReferenceError: ○○ is not defined	変数名・関数名の綴りの間違い。作っていない変数や関数を使っている
TypeError: Cannot read properties of undefined (reading 'y')	配列のない番号 (bullets[5] など) の .y を使おうとした。for の条件が <= になっている、break を忘れた、など
SyntaxError: Unexpected token '}'	{ } や () の数が合っていない。 , や ; の書き忘れ・書きすぎ
TypeError: Cannot read properties of null (reading 'getContext')	getElementById("game") の名前と、<canvas> の id が違う

- エラーが 1 つ出ると、そこから先の処理が実行されないことがある。上に表示されたエラーから順に直す。

(2) エラーが出ないのに、思いどおりに動かないとき

エラーが出なくても、条件の書き間違いなどで、思いどおりに動かないことがある。

- **コンソールに変数名を入力する**（第 26 回）：コンソールに `score` や `lives`、`enemies.length` などを入力して Enter キーを押すと、その時点の値が分かる。
- **`console.log` を一時的に書く**：たとえば、弾が当たったときに得点が増えているかを調べたいときは、`checkBulletHits` の `score += 100;` の下に `console.log(score);` を一時的に書く。当たるときに、コンソールに得点が表示される。確認したら、その行は消す。
- `update` の最初などに書くと、1 秒に 60 回表示されて読めなくなる。`if` の中など、たまにしか実行されない場所に書く。
- **1 行ずつ `//` で止めてみる**：あやしい行の先頭に `//` を付けて、動きがどう変わるかを見る（第 22 回・第 24 回・第 28 回の演習で行った方法）。

3. 見た目の調整

(1) 色の指定

`ctx.fillStyle` には、CSS と同じ書き方で色を指定する。

書き方	例	意味
<code>#rrggbb</code>	<code>"#ff4444"</code>	赤・緑・青の強さを、それぞれ 2 けた（00～ff）で書く
色の名前	<code>"white"</code> 、 <code>"orange"</code>	決まった名前の色
<code>rgba(赤, 緑, 青, 透明度)</code>	<code>"rgba(0, 0, 0, 0.6)"</code>	赤・緑・青は 0～255、透明度は 0（透明）～1（不透明）

- 背景を暗い色にして、自機・弾・敵・文字を明るい色にすると、見やすい。
- 自機と敵は、はっきり違う色にする。

(2) 複数の四角形で形を作る

`drawPlayer` の中で、`fillRect` を何回か使うと、四角形を組み合わせて、翼のある機体などの形にできる。

- 位置は、`player.x + 12` や `player.y + 20` のように、`player.x`・`player.y` をもとに計算する。こうすると、自機が動いても、形がくずれずに一緒に動く（第 21 回の発展と同じ考え方）。
- 当たり判定（`isHit`）は、これまでどおり `player` の `x`・`y`・`w`・`h` の四角形で行われる。描く形は、この四角形からはみ出さないようにすると、見た目と当たり判定がずれない。

演習

`exercise/main.js`（自分のシューティングゲーム）に書く。

演習 1 最終必須仕様に追いつく

- 講義内容 1 の表で、自分のゲームが 1~9 のどこまでできているかを確認する。
- できていない項目があれば、その回のレジメと `sample/main.js` を見て追加する。
- すべてできている人は、演習 2 へ進む。

演習 2 間違いを見つける

次の (1)~(4) には、それぞれ 1 か所ずつ間違いがある。どこが間違いで、ゲームがどうなるか（エラーが出るか、どんな動きになるか）を考えて、ノートに書く。分からないときは、`sample` をコピーしたファイルで、同じ間違いを実際を書いて確かめてもよい。

(1)

```
// 間違いのあるコード (1)
if (event.key === "arrowLeft") {
    isLeftPressed = true;
}
```

(2)

```
// 間違いのあるコード (2)
for (let i = 0; i <= bullets.length; i++) {
    ctx.fillRect(bullets[i].x, bullets[i].y, bullets[i].w, bullets[i].h);
}
```

(3)

```
// 間違いのあるコード (3)
function resetGame() {
    gameState = "play";
    score = 0;
    lives = 3;
    frameCount = 0;
    player.x = 184;
    bullets = [];
    shotTimer = 0;
}
```

(4)

```
// 間違いのあるコード (4)
function draw() {
    drawPlayer();
    drawBullets();
    drawEnemies();
    drawUI();
    if (gameState === "gameover") {
        drawGameOver();
    }
}
```

演習 3 見た目を調整する

- 背景・自機・弾・敵・文字の色を、自分で決めて変える。
- 変えたあとで、自機・弾・敵が見分けやすいかを確認する。

発展（任意）

- 自機を、複数の四角形を組み合わせた形で描く（胴体・翼・先端など）。
- 敵にも、目などの模様を描き足す（位置は `enemies[i].x`・`enemies[i].y` をもとに計算する）。
- 背景に、下へ流れる星を描く。ゲームの最初に、星のオブジェクト（`x`・`y`・`speed`）を `for` と `push` で 40 個作って配列に入れ、毎フレーム下へ動かす。画面の下に出た星は、上（`y` を 0）に戻す。